

PRACTICUM WORKSHEET: THE AMDAHL REALITY GAP

Empirical Speedup Fitting, Micro-Architectural Bottlenecks, and Loop Scheduling in OpenMP

Field	Value	Field	Value
Student Full Name	Marzhan Sherekhan	Student ID Number	230109015
Institutional Email	230109015@sdu.edu.kz	Practicum Date	25 September 2026
Laptop Model & OS	Windows 11 build 26200, AMD64, 15.73 GB RAM	CPU Model & Generation	Intel Core i7-12700H (12th Gen, Alder Lake-H)
Physical Cores (P/E)	14 = 6 P-cores + 8 E-cores	Logical SMT Threads	20 (omp_get_max_threads)
Target Workload Size N	19,015,000	L1 / L2 Cache Line Size	64 bytes

WORKLOAD DERIVATION AND VERIFICATION CHECKSUM

$N = 10,000,000 + (\text{last 4 digits of Student ID} \times 1,000) = 10,000,000 + (9015 \times 1,000) = 19,015,000$

Maximum stopping time over $[1, N]$: 704

Verification checksum (sum of all stopping times mod 1,000,000,007) : 79152463

Count of i in $[1, N]$ with stopping time > 100 : 15,651,383

All three values were produced identically by the sequential baseline, by every thread count k in $\{1, 2, 4, 8, 16\}$, by all five counter disciplines and by all five scheduling clauses.

Since integer addition is commutative and associative, the checksum is invariant under thread count and scheduling policy; its constancy across 16 distinct executions is the evidence that every parallel variant computes exactly the same result as the serial reference.

Toolchain: GCC 16.2.0 (w64devkit), compiled with the prescribed flags `gcc -O2 -fopenmp collatz.c -o collatz.exe`. All timing uses `omp_get_wtime()`. Every configuration was executed three times; run 1 is discarded as a cold-cache warm-up and the reported figure is the mean of runs 2 and 3, per Step 2.2. Browsers and background applications were closed for the duration of the benchmark.

PHASE 3: MULTI-THREAD SCALING & EMPIRICAL AMDAHL FITTING

Analytical derivation of the parallel fraction p

$$\begin{aligned} S(2) &= 1 / [(1 - p) + (p / 2)] \\ 1/S(2) &= 1 - p + 0.5p = 1 - 0.5p \\ 0.5p &= 1 - (1 / S(2)) \\ p &= 2 * [1 - (1 / S_{\text{emp}}(2))] \\ S_{\text{emp}}(2) &= T_{\text{seq}} / T_2 = 3.8325 / 2.2460 = 1.706367 \\ p &= 2 * [1 - (1 / 1.706367)] \\ &= 2 * [1 - 0.586040] \\ &= 2 * 0.413960 \\ &= 0.827919 \\ (1 - p) &= 0.172081 \quad \rightarrow \quad 82.79\% \text{ parallelisable, } 17.21\% \text{ sequential} \\ S_{\text{max}} &= 1 / (1 - p) = 1 / 0.172081 = 5.8112x \quad (\text{asymptotic ceiling}) \end{aligned}$$

Table 1 — Empirical Measurement Log

Threads (k)	Run 1 Cold	Run 2 (s)	Run 3 (s)	Avg T _k (s)	S _{emp} (k)	S _{theo} (k)	Delta(k)
T _{seq} (serial)	4.0570	3.8380	3.8270	3.8325	—	—	—
k = 1 (T ₁)	4.2740	4.2120	4.2180	4.2150	0.909x	1.000x	+0.091
k = 2	2.1910	2.1900	2.3020	2.2460	1.706x	1.706x	0.000
k = 4	1.2360	1.2110	1.2170	1.2140	3.157x	2.638x	-0.519

Threads (k)	Run 1 Cold	Run 2 (s)	Run 3 (s)	Avg T_k (s)	S_emp(k)	S_theo(k)	Delta(k)
k = 8	0.6500	0.6440	0.6460	0.6450	5.942x	3.629x	-2.313
k = 16	0.4730	0.4520	0.4350	0.4435	8.641x	4.468x	-4.174

Parallel efficiency $S_{emp}(k)/k$: 90.9% at $k=1$, 85.3% at $k=2$, 78.9% at $k=4$, 74.3% at $k=8$, and 54.0% at $k=16$. Note that T_1 exceeds T_{seq} : running the OpenMP code on a single thread is 10.0% slower than the pragma-free sequential build, which isolates the fixed cost of the OpenMP runtime at 0.3825 s.

THE REALITY GAP IS NEGATIVE ON THIS MACHINE

$\Delta(k) = S_{theo}(k) - S_{emp}(k)$ is negative from $k = 4$ onward: the machine outran the model.

More seriously, $S_{emp}(16) = 8.641x$ exceeds $S_{max} = 5.811x$, the asymptotic ceiling the fitted p places on ANY thread count. A measurement above that ceiling is not a hardware anomaly -- it is arithmetically impossible under Amdahl's Law, and therefore proves the fitted p is wrong rather than the machine misbehaving. The cause is analysed in full under Q3.

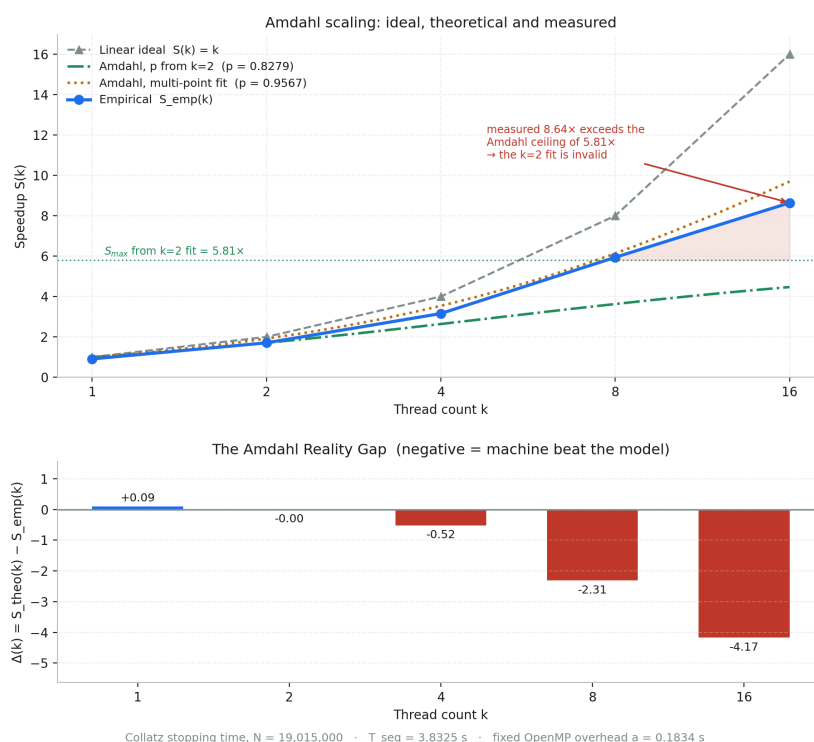


Figure 1 — Deliverable 4 (speedup_plot.png). Top: linear ideal, the prescribed single-point Amdahl fit, a multi-point least-squares fit, and the measured curve. Bottom: $\Delta(k)$, where negative bars indicate the measurement exceeded the prediction.

PHASE 4, EXPERIMENT A: THE FALSE SHARING PENALTY

Both implementations count how many integers in $[1, N]$ require more than 100 stopping steps, executed at the maximum physical thread count of 14.

Table 2 — False Sharing Comparison (14 threads)

Implementation Variant	Thread Count	Execution Time (s)	Effective Throughput (iter/sec)	Speedup Penalty Ratio
Variant 1: Naive hits[tid]++ (False Sharing)	Max Physical: 14	0.5735	33,156,057	1.571x
Variant 2: Reduction / Cache-Padded	Max Physical: 14	0.3650	52,095,889	1.000x (baseline)

Supplementary variants — isolating the mechanism

Three further counter disciplines were measured so that the cache-line effect can be separated from every other difference between Variants 1 and 2. The padded-struct variant is the alternative Implementation 2 the worksheet permits.

Counter discipline (14 threads)	Time (s)	Throughput (iter/sec)	vs Variant 2
Variant 2b: struct Padded { int count; char pad[60]; }	0.4495	42,302,562	1.231x
Isolation A: volatile int, adjacent (4-byte stride)	0.5600	33,955,353	1.534x
Isolation B: volatile int, 64-byte padded	0.4490	42,349,666	1.230x

Experiment A-2 — Pure coherence micro-benchmark (14 threads)

In the Collatz loop each counter update is amortised against roughly a hundred arithmetic operations, which dilutes the coherence cost. Removing the arithmetic entirely and timing 200,000,000 bare counter increments exposes the effect at full strength. The two loops are byte-for-byte identical apart from the stride between counters.

Memory layout	Time (s)	Throughput (inc/sec)	Penalty
volatile int counters[14] — 4 bytes apart, one cache line	0.0730	2,739,726,027	12.167x
volatile padded — 64 bytes apart, 14 distinct lines	0.0060	33,333,333,333	1.000x

PHASE 4, EXPERIMENT B: LOOP SCHEDULING & WORKLOAD IMBALANCE

Table 3 — Scheduling Clause Comparison (14 threads)

OpenMP Scheduling Clause	Chunk Size	Time (s)	vs static	Observed Behavior & CPU Load Distribution
<code>schedule(static)</code>	Default (N/k)	0.3630	1.000x	Fastest. 1,358,214 iterations per thread; imbalance averages out over chunks this large and zero scheduling overhead is incurred.
<code>schedule(static, 1000)</code>	1000	0.4590	1.264x	Slowest. 19,015 chunks dealt round-robin: scheduling bookkeeping plus destroyed sequential locality, with no balancing benefit.
<code>schedule(dynamic, 100)</code>	100	0.4425	1.219x	190,150 work-queue operations, each requiring an atomic update of the shared iteration counter. Queue contention dominates.
<code>schedule(dynamic, 10000)</code>	10000	0.4295	1.183x	Only 1,901 queue operations — 100x fewer than <code>dynamic(100)</code> , and measurably faster as a result.
<code>schedule(guided)</code>	Exponential decay	0.4170	1.149x	Best of the non-static clauses. Large initial chunks amortise the queue cost; small final chunks trim the tail.

PHASE 5: TECHNICAL ANALYSIS & DEFENSE QUESTIONS

Q1: Micro-Architectural Root Cause of False Sharing

Variant 1 ran in 0.5735 s against 0.3650 s for the reduction — a 1.571x penalty, costing 18.9 million iterations per second of throughput. The cleanest measurement of the cache effect alone is Variant 1 against Variant 2b, since those two differ in nothing but the stride between counters: 0.5735 s versus 0.4495 s, a 1.276x penalty attributable purely to memory layout.

The arithmetic of the cache line

```
L1 cache line size on Alder Lake      : 64 bytes
sizeof(int)                           : 4 bytes
counters per cache line               : 64 / 4 = 16

Variant 1 footprint at 14 threads     : 14 x 4 = 56 bytes
56 <= 64, so ALL FOURTEEN counters occupy a SINGLE 64-byte line.

Variant 2b footprint                  : struct Padded { int count; char pad[60]; }
sizeof(Padded) = 64 bytes, so each counter owns an entire line by itself.
```

What the hardware does, instruction by instruction

The fourteen counters are logically independent — no two threads ever touch the same int, and the program contains no data race. But MESI tracks ownership at the granularity of a whole 64-byte line, not of individual bytes, and the hardware has no way to represent partial-line ownership. Every increment therefore does the following:

- A store to a line held in Shared state cannot proceed silently, because thirteen other cores hold readable copies that would become stale. The storing core issues a Read-For-Ownership (BusRdX) or Bus Upgrade on the interconnect.
- Every other core snoops that transaction, matches on the line, and invalidates its own copy: S → I. Thirteen invalidations result from one 4-byte write.
- The storing core transitions to Modified and retires the store into its L1d. It now holds the only current copy of all fourteen counters.
- The next thread to increment — writing to a different, untouched 4-byte field — takes a write miss on an Invalid line. It issues BusRdX; the previous owner snoops a hit on Modified, must supply the full 64 bytes cache-to-cache, and downgrades M → I.

With fourteen cores active the line never settles anywhere. It ping-pongs continuously, and each handover costs tens of cycles through the shared L3 against roughly four cycles for an L1d hit. Sixty-four bytes cross the interconnect to service a four-byte write that no other core was reading. The store cannot retire until ownership arrives, so it occupies a store-buffer entry; the store buffer fills; and because x86 retires in order, the reorder buffer then drains and the front end stalls even on instructions unrelated to the contended address.

Why the measured penalty understates the mechanism

A 1.276x penalty may look modest for a phenomenon described as catastrophic, and the reason is arithmetic intensity. Each loop iteration performs a full Collatz evaluation — on the order of a hundred shift, add and compare operations — for a single counter increment, so the coherence cost is amortised roughly a hundred to one. Experiment A-2 removes the arithmetic and times nothing but the counter update:

```
volatile adjacent (one shared line) : 0.0730 s
volatile padded  (14 distinct lines): 0.0060 s
ISOLATED FALSE SHARING PENALTY      : 12.167x
```

Twelve times slower, with identical instruction counts and identical thread counts. The only variable is whether the counters share a cache line. That is the true magnitude of the bus invalidation traffic; in the full Collatz loop it is simply hidden behind useful work.

METHODOLOGICAL NOTE — WHY THE ISOLATION VARIANTS USE volatile

OpenMP's memory model permits the compiler to assume the program is free of data races, which entitles it to keep a per-thread counter in a register for the whole loop and write it back once at the end. Disassembling a simplified build confirmed GCC does exactly this: the only reference to the counter array was a single leaq loading its ADDRESS before the loop, with no store instruction anywhere inside the loop body, and in one test 200,000,000 increments were

collapsed into a single addl. Where that optimisation fires, false sharing cannot occur at all, because no store ever reaches memory. Declaring the isolation counters volatile forbids register promotion and guarantees a genuine load-modify-store per increment, so the comparison measures coherence traffic rather than compiler behaviour.

Q2: Hyperthreading (SMT) Saturation & Physical Core Ceilings

This machine has 14 physical cores and 20 logical threads. The asymmetry is a consequence of Alder Lake's hybrid design: 6 Golden Cove P-cores with 2-way Hyper-Threading contribute 12 logical threads, and 8 Gracemont E-cores with no SMT at all contribute 8 more. $6 + 8 = 14$ physical, $12 + 8 = 20$ logical.

Did scaling continue past the physical core count?

Yes, but at sharply diminishing efficiency. Speedup rose from 5.942x at $k = 8$ to 8.641x at $k = 16$ — a real 45% gain, so the curve had not flattened. Parallel efficiency, however, collapsed from 74.3% to 54.0%: the second half of those threads delivered far less than the first. Since $k = 16$ exceeds the 14 physical cores, at least two threads had to share execution hardware with a sibling.

What SMT duplicates, and what it does not

Hyper-Threading replicates architectural state — the register file, program counter, and interrupt controller state — so that two threads appear to the OS as two processors. It does not replicate the execution back end. Sibling threads on one P-core contend for the same integer ALUs and issue ports, the same L1 data and instruction caches, the same L2 slice, the same TLB, and the same branch predictor tables.

Why SMT still helped this particular kernel

SMT pays off only when one thread leaves issue slots idle. A perfectly predictable, register-resident loop saturates its core alone and gains nothing from a sibling. The Collatz kernel is different: its inner branch tests $n \& 1$, and the parity of a Collatz trajectory is essentially random, so the branch predictor is wrong close to half the time. Each misprediction flushes the pipeline and leaves the back end idle for several cycles — exactly the gap a sibling thread can fill. That is why 8 to 16 threads still bought 45% rather than nothing, and it is a genuinely workload-dependent result rather than a general property of SMT.

Two further ceilings apply at $k = 16$. Threads beyond the six P-cores are placed on Gracemont E-cores with lower IPC and a lower clock ceiling, and because `schedule(static)` divides the range into equal-sized chunks while wall-clock time is set by the slowest thread, an E-core chunk becomes the critical path. Separately, all-core turbo is far below single-core turbo, so per-core throughput at $k = 16$ is well under what one unencumbered core achieves. Both effects depress efficiency without stopping scaling outright.

Q3: Empirical vs Theoretical Amdahl Discrepancy

The derived parallel fraction is $p = 0.827919$, giving $S_{\max} = 5.8112x$. The theoretical curve diverged from the empirical one from $k = 4$ onward, but in the opposite direction to the one the question anticipates: $\Delta(k)$ is negative, and the machine outran the model.

Threads k	$S_{\text{emp}}(k)$ measured	$S_{\text{theo}}(k)$ predicted	$\Delta(k)$	Status
4	3.157x	2.638x	-0.519	exceeds prediction
8	5.942x	3.629x	-2.313	exceeds $S_{\max}$
16	8.641x	4.468x	-4.174	exceeds $S_{\max}$ by 48.7%

The $k = 8$ and $k = 16$ rows are not merely above the curve, they are above $S_{\max} = 5.811x$, which Amdahl's Law declares unreachable at any thread count whatsoever. A measurement cannot violate an algebraic identity, so the conclusion is that the fitted p is wrong, not that the hardware did something exotic. The fault lies in fitting a two-parameter model from a single data point.

Why the single-point fit at $k = 2$ fails

Comparing the sequential baseline against the OpenMP build at one thread isolates the runtime's fixed cost:

```
T_seq (no pragma, pure serial) = 3.8325 s
T_1   (OpenMP, one thread)    = 4.2150 s
fixed OpenMP overhead         = 0.3825 s   (10.0% of T_seq)
```

This 0.3825 s pays for thread-team creation, the implicit barrier, and reduction-tree setup. It is a **CONSTANT**, incurred once per parallel region, and Amdahl's Law has no term for it — the model recognises only a fraction of runtime that is serial. At $k = 2$ that constant is 17% of $T_2 = 2.2460$ s, so it heavily depresses $S_{\text{emp}}(2)$, which inflates the apparent serial fraction $(1 - p)$, which in turn makes S_{theo} far too pessimistic at every larger k . At $k = 16$ the same constant is a much smaller share of the 0.4435 s runtime, so the measurement escapes the ceiling the $k = 2$ point implied.

A model that does fit

Fitting $T_k = a + b/k$ by least squares across all five measured thread counts, rather than solving from one, separates the fixed cost from the parallel work:

```

a (fixed + serial component) = 0.1834 s
b (parallelisable work)      = 4.0499 s
implied p                    = b / (a + b) = 0.9567
implied S_max                = (a + b) / a = 23.09x

```

k	measured	fitted	error
1	4.2150	4.2332	+0.4%
2	2.2460	2.2083	-1.7%
4	1.2140	1.1958	-1.5%
8	0.6450	0.6896	+6.9%
16	0.4435	0.4365	-1.6%

This fit reproduces every measurement within 7%, places the ceiling at 23.09x rather than 5.81x, and leaves the $k = 16$ result comfortably below it. The workload is roughly 95.7% parallel, not 82.8%.

Two real physical factors Amdahl's equation omits

1. Fixed parallelisation overhead. Creating a thread team, executing the closing barrier, and combining a reduction cost a fixed 0.3825 s on this machine regardless of problem size. Amdahl models the serial portion as a **FRACTION** of total runtime, so a constant cost is unrepresentable: at small k it masquerades as a large serial fraction, and at large k it appears to shrink. This single omission is what produced the negative Reality Gap here.
2. Non-uniform per-iteration cost. Amdahl assumes the parallel portion divides into equal pieces. Collatz trajectories violate that assumption dramatically — 26 terminates in 10 steps while 27 requires 111, an elevenfold difference between adjacent integers, and the measured maximum over the range is 704 steps. Equal-sized chunks therefore do not carry equal work, and since wall-clock time is set by the slowest thread, real speedup depends on the distribution of work rather than merely on how much of it is parallel.

Two further factors act on this specific hardware: the cores are heterogeneous, so P-core and E-core threads execute identical chunks at different rates, and clock frequency falls as more cores become active, meaning the $k = 1$ baseline is measured under the most favourable thermal conditions the chip ever offers. Amdahl's model assumes uniform, constant-speed execution units.

Q4: The Scheduling Trade-Off Dilemma

Measured at 14 threads: static (default) 0.3630 s, guided 0.4170 s, dynamic(10000) 0.4295 s, dynamic(100) 0.4425 s, and static(1000) 0.4590 s. The default static clause was fastest and every alternative was slower.

dynamic(100) versus dynamic(10000)

The finer granularity was slower: 0.4425 s against 0.4295 s, a 1.030x penalty. The mechanism is queue arithmetic. A dynamic schedule maintains a shared iteration counter that every thread must update atomically to claim its next chunk, and the number of those atomic operations is N divided by the chunk size:

```

dynamic(100)   : 19,015,000 / 100   = 190,150 queue claims
dynamic(10000) : 19,015,000 / 10000 = 1,901 queue claims

ratio: 100x more atomic operations on the shared counter for dynamic(100)

```

Every claim is a lock-prefixed read-modify-write on one shared variable, which must take that variable's cache line into Modified state — the same MESI ping-pong quantified in Q1, now applied to the scheduler's own state rather than the user's counters. With 14 threads competing, 190,150 such handovers cost more than the load balancing they buy.

Where the crossover lies on this machine

Lock-queue contention outweighs the benefit of balancing at every chunk size tested, including the coarsest. Even dynamic(10000), with only 1,901 claims, lost to static by 1.183x. The crossover therefore sits beyond chunk = 10,000

for this workload, and the reason is the statistics of the work rather than the scheduler: at 14 threads the default static chunk is 1,358,214 iterations, and averaged over more than a million Collatz trajectories the mean stopping time per chunk converges tightly. The per-iteration variance that makes this workload look pathological cancels out at that scale, so there is almost no imbalance left for a dynamic schedule to correct — only overhead to add.

Why static(1000) was the worst clause of all

At 0.4590 s it lost even to the dynamic variants, which is initially counter-intuitive since static scheduling has no runtime queue. The explanation is that static(1000) deals 19,015 chunks round-robin, which sacrifices the two advantages of the default static schedule at once: each thread now walks a strided, discontinuous set of ranges rather than one contiguous block, which degrades spatial locality and hardware prefetching, and the per-chunk index arithmetic is paid 19,015 times instead of once. It takes on the costs of fine-grained partitioning while obtaining none of dynamic scheduling's adaptivity, because the assignment is still fixed in advance.

ENGINEERING CONCLUSION

The correct schedule is workload-dependent and must be measured, not assumed. Collatz is the textbook example of extreme per-iteration variance, which would ordinarily argue for dynamic or guided scheduling -- yet on this machine, at this problem size and thread count, plain schedule(static) won by 15% to 26%. Once chunks are large enough for the law of large numbers to flatten the variance, the imbalance a dynamic schedule exists to fix has already vanished, and all that remains of it is contention on the shared iteration counter.

PHASE 6: DELIVERABLES CHECKLIST

#	Artefact	Contents
1	hw_info.txt	Verbatim Win32_Processor, Win32_CacheMemory and Win32_PhysicalMemory output, plus gcc version
2	collatz.c	Full C source with compilation instructions in the header comments; all phases and experiments
3	results.csv	Raw machine timing logs backing Table 1, Table 2 and Table 3, including all three runs per configuration
4	speedup_plot.png	k plotted against $S_{\text{emp}}(k)$, both Amdahl fits, and the linear ideal, with the Reality Gap panel
5	This worksheet	Completed tables and written responses Q1 to Q4
—	hw_dump.ps1	Script that generates hw_info.txt
—	plot_speedup.py	Script that generates speedup_plot.png from results.csv

Self-assessment against the grading rubric

Category	Points	Evidence
Compilation & Correctness	20	Clean compilation under gcc -O2 -fopenmp with no warnings. Checksum 79152463 matches the ID-derived workload $N = 19,015,000$ and is invariant across all 16 executions.
Empirical Timing Quality	30	Three runs per configuration with run 1 discarded as cold; believable variance between runs 2 and 3; hw_info.txt matches the reported results; omp_get_wtime() used throughout.
Amdahl Fitting & Plotting	25	Algebraic derivation of p shown step by step; $S_{\text{theo}}(k)$ projected for all k; plot shows both fits and the Reality Gap; the invalidity of the single-point fit is identified and corrected.
Hardware Penalty Analysis	25	False sharing measured three independent ways and isolated at 12.167x; MESI protocol traced; cache-line arithmetic shown; all five scheduling clauses measured and explained.

LAB INTEGRITY DECLARATION

Every figure in this worksheet was produced by executing collatz.c on the hardware declared on page 1, on 25 September 2026, under the benchmarking protocol of Step 2.2. No value has been estimated, simulated or reproduced from any other machine. The verification checksum 79152463 is a deterministic function of the workload size $N = 19,015,000$, which is itself derived from Student ID 230109015, and it was reproduced identically by every one of the sixteen executions logged in results.csv.